

Intro to Deep Learning

Lecture 07: Transfer Learning With Pretrained CNNs

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. Why use pretrained CNNs?
2. What is a backbone?
3. What is a classifier head?
4. What is the difference between freezing and fine-tuning?
5. Why do pretrained models need specific transforms?
6. Why do we use smaller learning rates for fine-tuning?

Lecture rhythm: pretrained features -> frozen baseline -> fine-tuning -> transforms

Recap From Lecture 06

To train stronger CNNs, we used:

```
BatchNorm  
augmentation  
residual connections  
experiment comparison
```

Today:

```
instead of training a CNN from scratch,  
reuse a CNN trained on a large dataset
```

Part 1

Why transfer learning?

Training From Scratch Is Expensive

Large CNNs need:

- many images
- many training steps
- careful optimization
- substantial compute
- good regularization

For a small course dataset, training a large CNN from scratch is often wasteful.

Transfer learning gives us a shortcut.

Transfer Learning

Transfer learning means:

use knowledge learned on one task
to help solve another task

For CNNs:

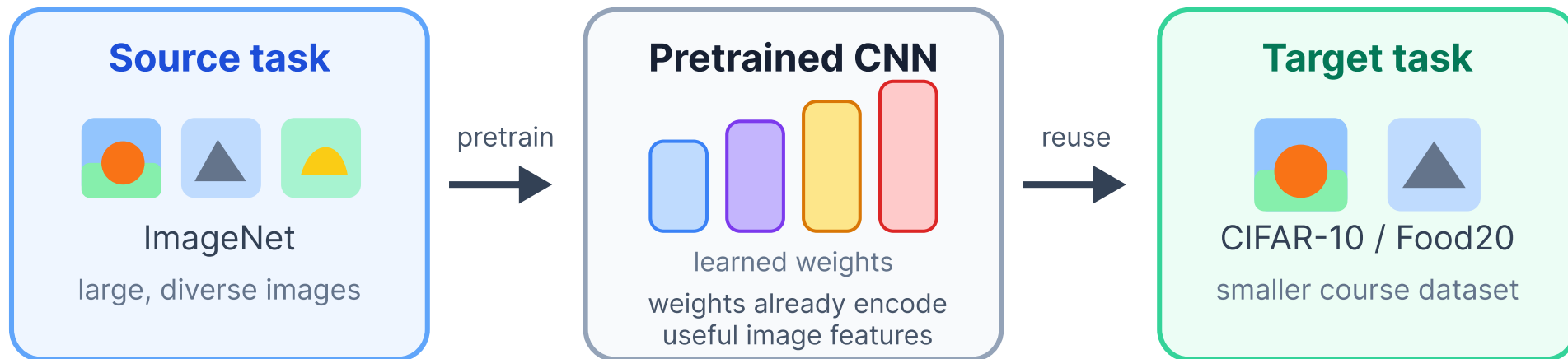
pretrain on a large image dataset
reuse on a smaller target dataset

Example:

ImageNet → CIFAR-10

ImageNet To A Course Dataset

Transfer learning reuses visual representations



What Did The CNN Learn?

A pretrained CNN has learned useful visual features.

Early layers:

edges, colors, simple textures

Middle layers:

parts, motifs, shapes

Later layers:

task-specific class features

Why Transfer Works

Many image datasets share low-level visual structure.

Examples:

```
edges  
textures  
color contrasts  
object parts
```

So a pretrained model often starts from a better representation than random weights.

This is especially helpful with limited data.

Part 2

Backbone, head, and freezing

Backbone And Head

A CNN classifier can be split into:

```
backbone -> head
```

Backbone:

```
extracts features from the image
```

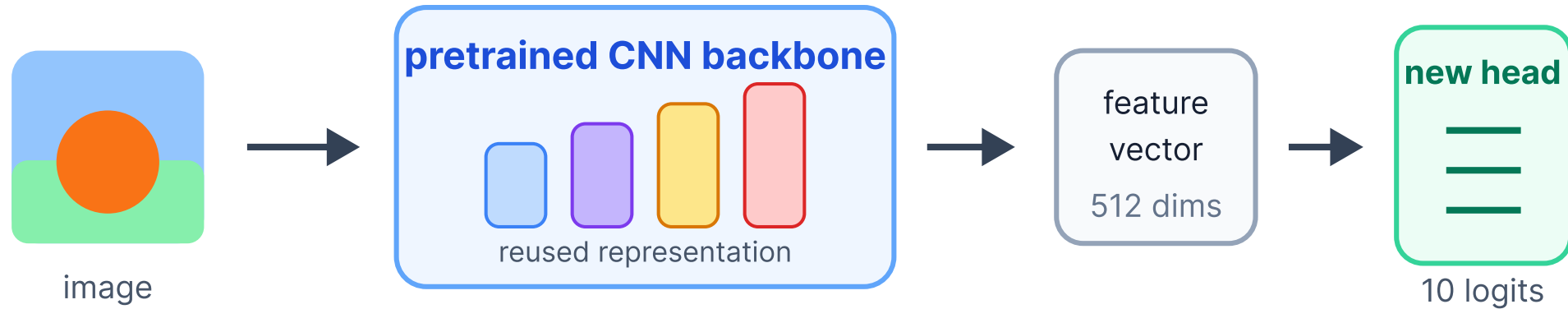
Head:

```
maps features to task-specific logits
```

Transfer learning usually keeps the backbone and replaces the head.

Backbone And Head: Picture

Backbone extracts features, head predicts classes



ResNet18 In This Course

In Seminar 07, you will use:

```
resnet18(weights=ResNet18_Weights.DEFAULT)
```

The model was pretrained for ImageNet classification.

Original output:

```
1000 ImageNet class logits
```

Our target:

```
10 CIFAR-10 class logits
```

Replace The Classifier Head

ResNet18 final layer:

```
model.fc
```

For CIFAR-10:

```
num_features = model.fc.in_features  
model.fc = nn.Linear(num_features, 10)
```

Now the model outputs:

```
[B, 10]
```

Same `CrossEntropyLoss` as before.

Frozen Feature Extractor

Frozen transfer learning:

```
keep pretrained backbone fixed  
train only the new head
```

Why do this?

- faster training
- fewer trainable parameters
- lower overfitting risk
- good first baseline

This is often the safest starting point.

Freezing Parameters

In PyTorch:

```
for parameter in model.parameters():  
    parameter.requires_grad = False
```

Then unfreeze the head:

```
for parameter in model.fc.parameters():  
    parameter.requires_grad = True
```

The optimizer should receive only trainable parameters.

Mini-Check: Freezing Bug

Which setup is wrong for a frozen ResNet18 baseline?

```
# A
for p in model.parameters():
    p.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 10)
optimizer = AdamW(model.fc.parameters(), lr=1e-3)
```

```
# B
model.fc = nn.Linear(model.fc.in_features, 10)
for p in model.parameters():
    p.requires_grad = False
optimizer = AdamW(model.fc.parameters(), lr=1e-3)
```

Find the bug. What happens during training?

Answer

Setup **B** is wrong.

It creates the new classifier head, then freezes the whole model:

```
model.fc = nn.Linear(model.fc.in_features, 10)
for p in model.parameters():
    p.requires_grad = False
```

So the new head is frozen too.

The optimizer receives `model.fc.parameters()`, but those parameters do not receive gradients.

What Freezing Means

Frozen parameters:

used in the forward pass
not updated by the optimizer

Trainable parameters:

used in the forward pass
receive gradients
updated by the optimizer

Freezing changes learning, not inference computation.

Fine-Tuning

Fine-tuning means:

```
start from pretrained weights  
continue updating some or all of them  
on the target dataset
```

This can adapt features to the new domain.

But it is riskier than freezing:

- more trainable parameters
- more overfitting risk
- easier to damage useful pretrained features

Part 3

Fine-tuning safely

Partial Fine-Tuning

A common compromise:

```
freeze early layers  
fine-tune later layers and the head
```

Why?

Early layers are general:

```
edges, colors, textures
```

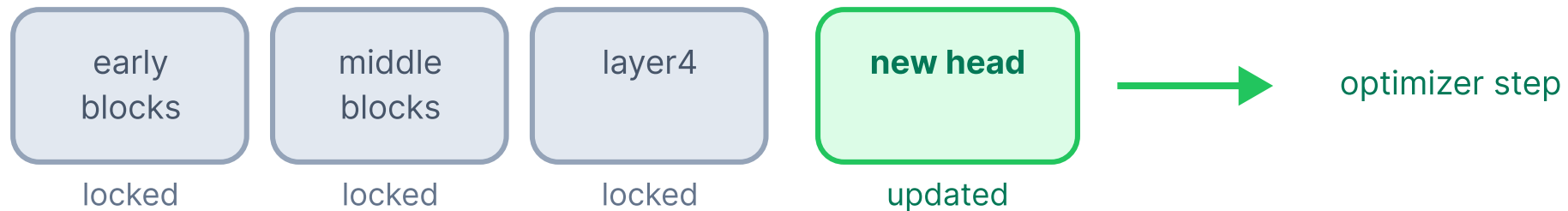
Later layers are more task-specific:

```
object parts and class-relevant features
```

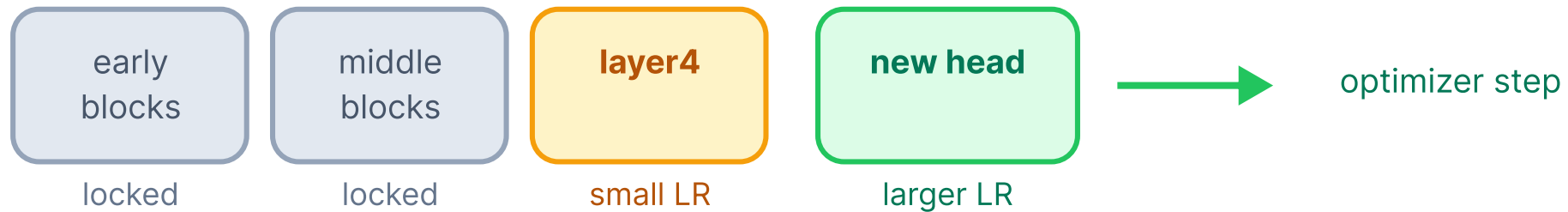
Frozen vs Partially Fine-Tuned

Freezing changes which weights are updated

Frozen feature extractor



Partial fine-tuning



Learning Rates

For training a new head:

larger LR is usually okay

For pretrained layers:

use smaller LR

Reason:

pretrained weights are already useful
we want to adapt them gently

Too high LR can destroy useful representations.

Different Learning Rates

Common fine-tuning recipe:

```
head:      higher LR  
backbone: lower LR
```

Example:

```
optimizer = AdamW([  
    {"params": model.layer4.parameters(), "lr": 1e-5},  
    {"params": model.fc.parameters(), "lr": 1e-4},  
])
```

Use this idea when the codebase keeps parameter groups simple enough.

Part 4

Preprocessing and dataset shift

Pretrained Transforms Matter

Pretrained models expect inputs similar to pretraining.

That includes:

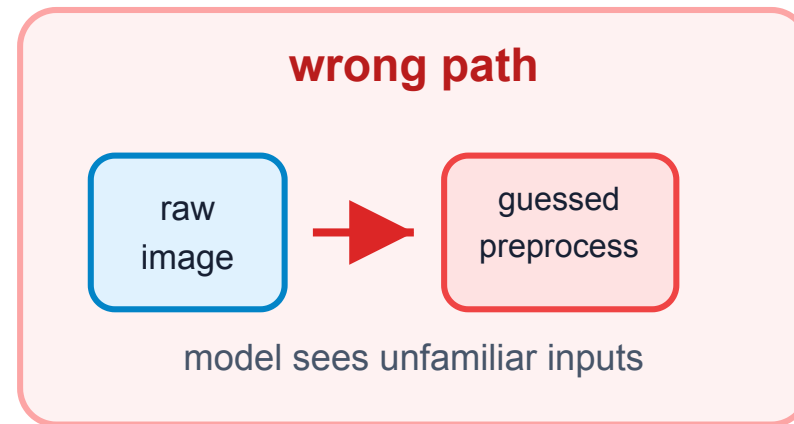
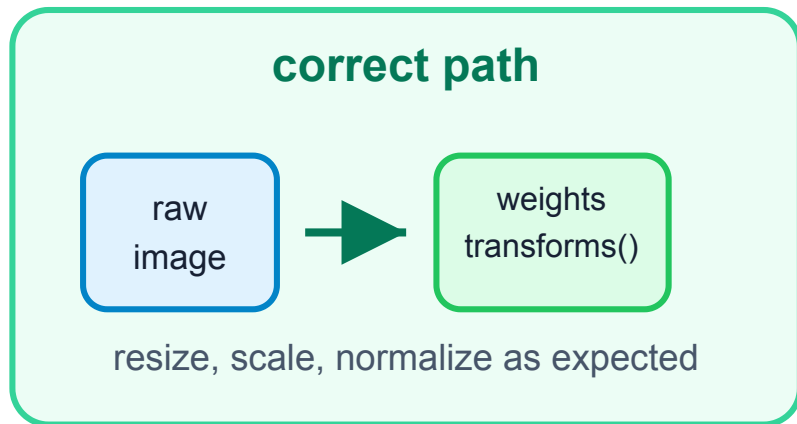
- image size
- channel order
- pixel scaling
- normalization mean
- normalization standard deviation

If preprocessing is wrong, the model receives images in an unfamiliar format.

Performance can drop sharply.

Transforms Matter: Picture

Weights and preprocessing belong together



Official torchvision Transforms

Torchvision weights include their expected transforms.

For example:

```
weights = ResNet18_Weights.DEFAULT  
transform = weights.transforms()
```

This is safer than guessing preprocessing manually.

Important principle:

```
model weights and preprocessing belong together
```

Mini-Check: Transform Debugging

You fine-tune ResNet18 and see:

- train accuracy improves quickly
- validation accuracy stays close to random
- loss is noisy across validation batches

Code:

```
train_transform = weights.transforms()  
val_transform = transforms.ToTensor()
```

Question:

What is suspicious here?
What would you check first before changing the model?

Answer

The suspicious part is the transform mismatch:

```
train_transform = weights.transforms()  
val_transform = transforms.ToTensor()
```

The validation images do not use the preprocessing expected by the pretrained weights.

First check:

```
resize / crop  
pixel scaling  
normalization mean and std
```

before changing architecture or learning rate.

Dataset Shift

Transfer learning works best when source and target are related.

Source:

ImageNet natural images

Target:

CIFAR-10 natural images

This is a reasonable match.

For very different data, pretrained features may help less.

ImageNet Bias

ImageNet pretraining is powerful, but not neutral.

The model has seen:

- certain object categories
- certain image styles
- certain collection biases
- high-resolution natural images

For medical, satellite, industrial, or artistic images, check transfer carefully.

Do not assume pretrained always means best.

Part 5

Embeddings and experiment comparison

Embeddings

The backbone output can be viewed as an embedding.

Embedding:

```
a vector representation of an input
```

For images:

```
similar images should often have similar embeddings
```

This idea will return in visual search and CLIP.

Backbone As Embedder

Remove the head to use the backbone as an embedder



Feature Extractor vs Classifier

With a classifier head:

```
image -> backbone -> feature vector -> logits
```

Without the classifier head:

```
image -> backbone -> feature vector
```

The second version is useful for:

- retrieval
- clustering
- nearest-neighbor search
- visualization

Fair Comparison Template

Compare models by behavior, not only final accuracy

model	trainable params	val acc	time	notes
small CNN baseline	small	?	fast	from scratch
frozen ResNet18	head only	?	fast	low risk
partial fine-tune	layer4 + head	?	slower	higher ceiling

When Freezing Is Better

Prefer freezing when:

- target dataset is small
- compute is limited
- overfitting is severe
- source and target are similar
- you need a quick baseline

Frozen features can be surprisingly strong.

They are also easier to debug.

When Fine-Tuning Is Better

Prefer fine-tuning when:

- you have enough data
- target domain differs from pretraining
- frozen features are not good enough
- you can afford slower training
- you can tune learning rate carefully

Fine-tuning usually has higher ceiling and higher risk.

Common Mistakes

Mistake 1:

`forgetting to replace model.fc`

Mistake 2:

`freezing everything, including the new head`

Mistake 3:

`using the wrong transforms`

Mistake 4:

`learning rate too high during fine-tuning`

Debugging Checklist

Before training, check:

- only intended parameters are trainable
- optimizer sees trainable parameters
- preprocessing comes from the weights object
- training and validation use compatible transforms
- `model.train()` and `model.eval()` are used correctly

This prevents most transfer-learning bugs.

Seminar Bridge

In Seminar 07, you will:

1. load pretrained ResNet18 weights
2. inspect the official transforms
3. replace the final classifier layer
4. train a frozen feature extractor
5. fine-tune `layer4 + fc`
6. compare frozen, fine-tuned, and small-CNN results

Main idea:

pretrained features are a strong starting point

Takeaways

Transfer learning reuses useful representations.

Key decisions:

- which model to start from
- which preprocessing to use
- which layers to freeze
- which layers to fine-tune
- what learning rates to use

Next lecture:

attention and transformers